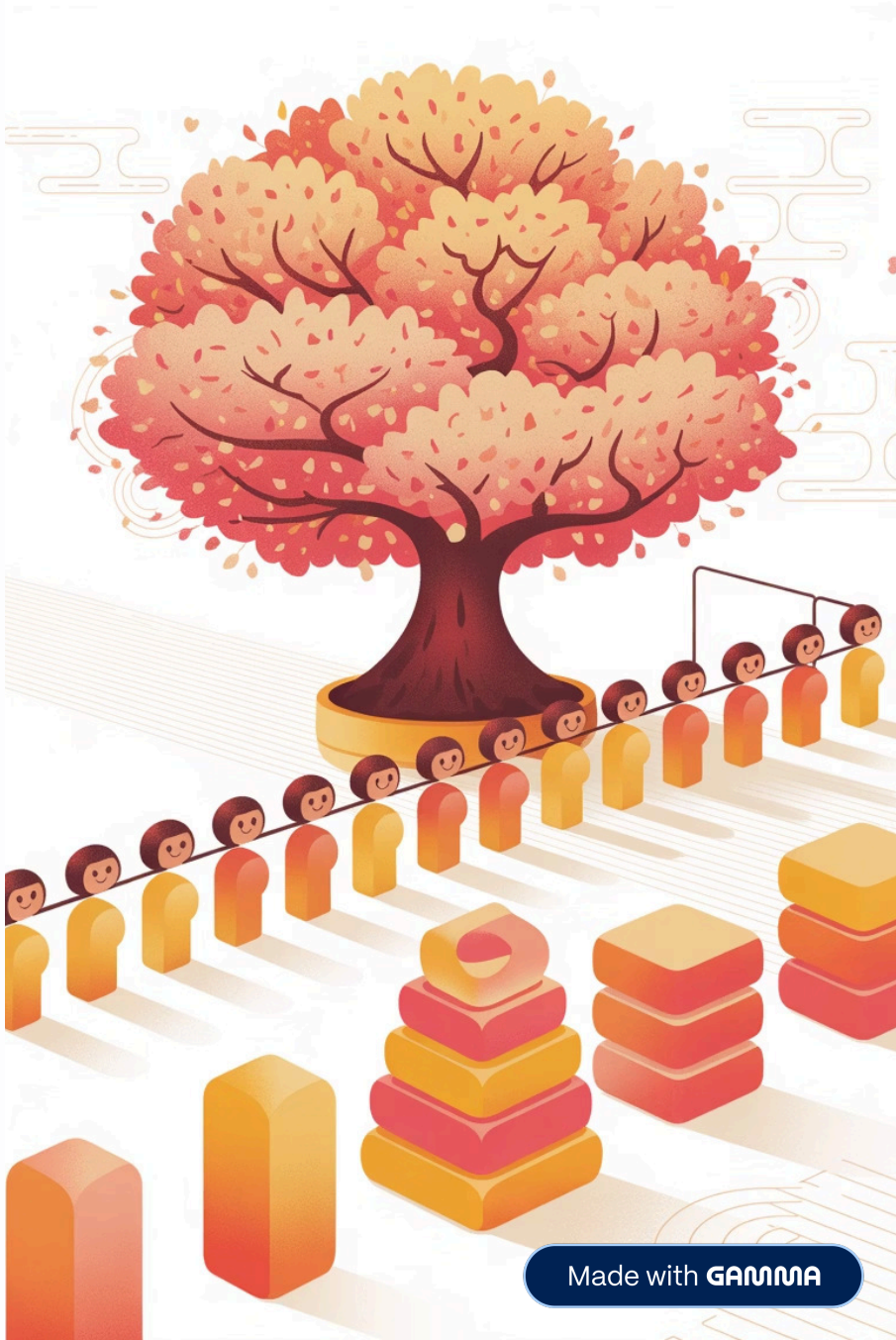
Listas y Tuplas: Colecciones de Datos Ordenados

Estructuras esenciales para manejar colecciones de información en análisis de datos con Python.



Listas

Estructuras Dinámicas y Modificables

Sintaxis: Se definen con corchetes `[]`

```
lista_clientes = ["Ana", "Beto", "Carlos"]
```

Mutabilidad (CLAVE): Las listas se pueden modificar después de su creación. Puedes añadir, eliminar o cambiar elementos en cualquier momento.

Uso en Análisis de Datos: Ideal para el flujo de trabajo de limpieza y pre-procesamiento (ETL), donde los datos cambian constantemente.

Ejemplo: Agregar nuevas filas de datos o eliminar valores atípicos (outliers).

Made with GAMMA

Listas: Manipulación I - Añadir y Extender

01

.append(elemento)

Agrega un elemento al final de la lista.

```
lista.append("Nuevo")
```

02

.insert(índice, elemento)

Agrega un elemento en una posición específica.

```
lista.insert(0, "Inicio")
```

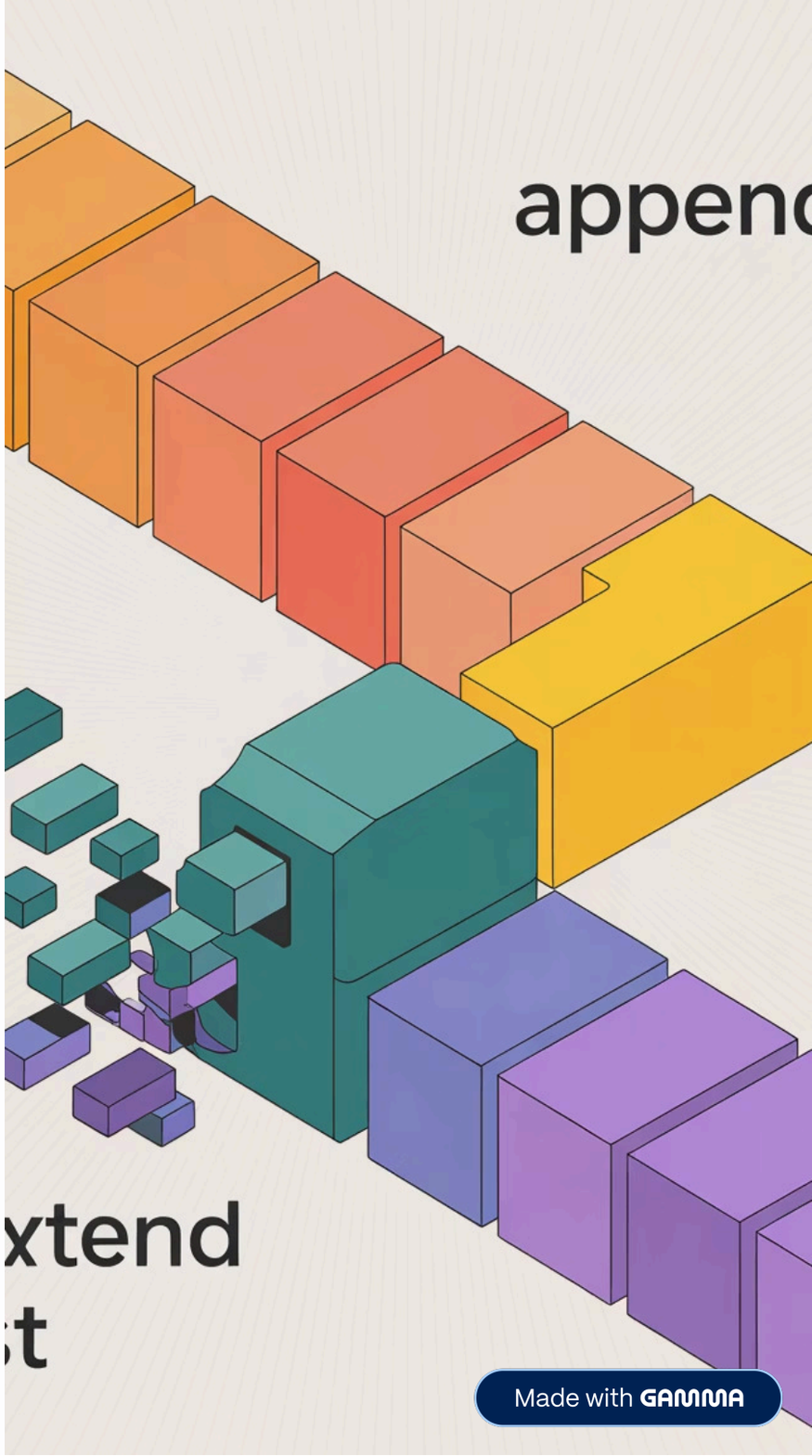
03

.extend(otra_lista)

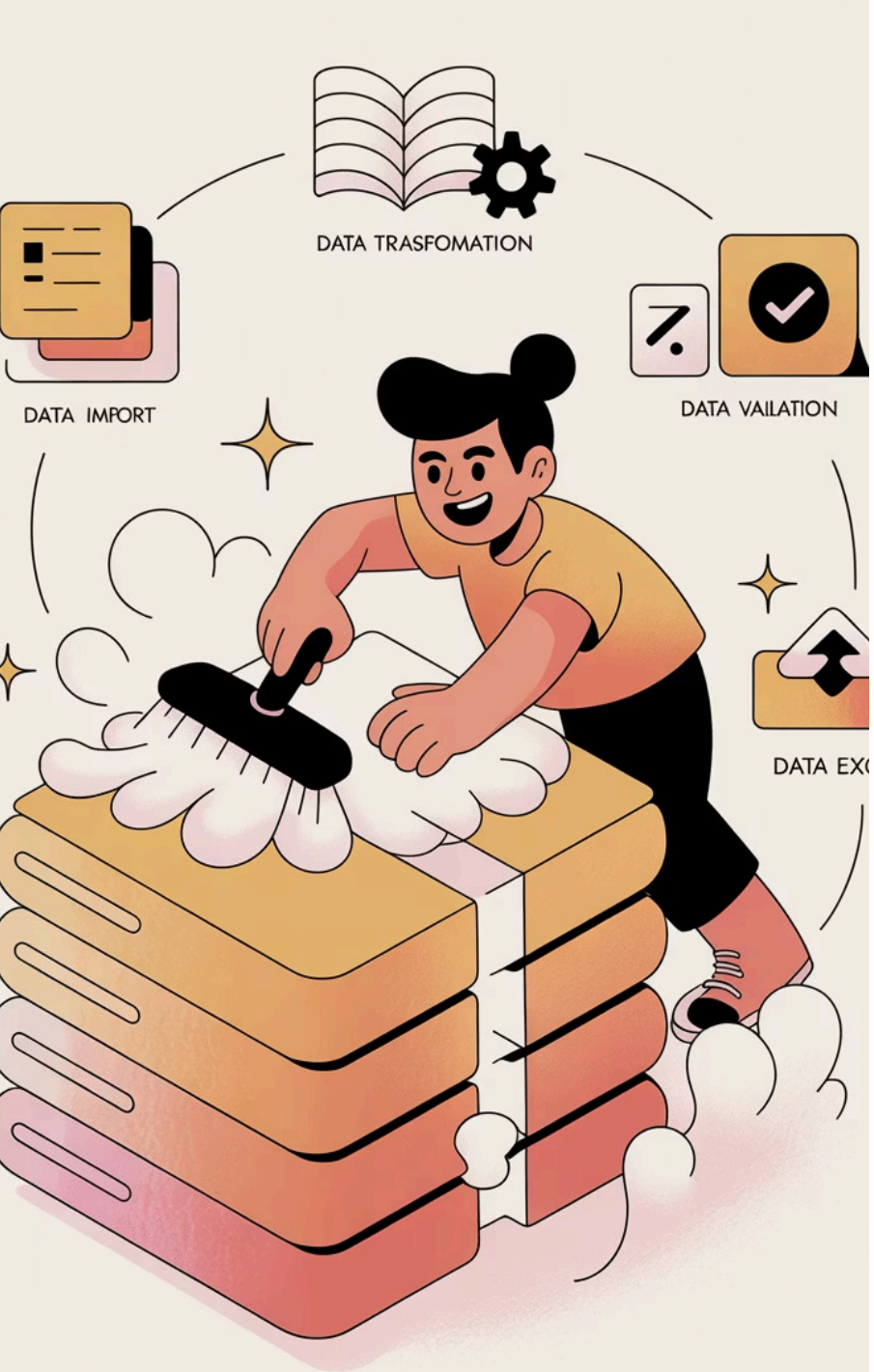
Agrega todos los elementos de otra lista al final, fusionando colecciones.

```
lista.extend(otra_lista)
```

Tip Análisis de Datos: Usa `extend()` o el operador `+` para consolidar listas de datos de múltiples fuentes.



Made with GAMMA



Listas: Manipulación II - Eliminar y Ordenar

.remove(valor)

Elimina la primera aparición de un elemento por su valor específico.

.pop(índice)

Elimina y devuelve el elemento en una posición específica. Ideal para extraer y usar un valor.

.clear()

Elimina todos los elementos de la lista, dejándola completamente vacía.

.sort()

Ordena los elementos de la lista in-place (modifica la lista original).

Made with GAMMA

Listas: Propiedades para Análisis Descriptivo

Estadísticas Básicas de una Colección

len(lista)

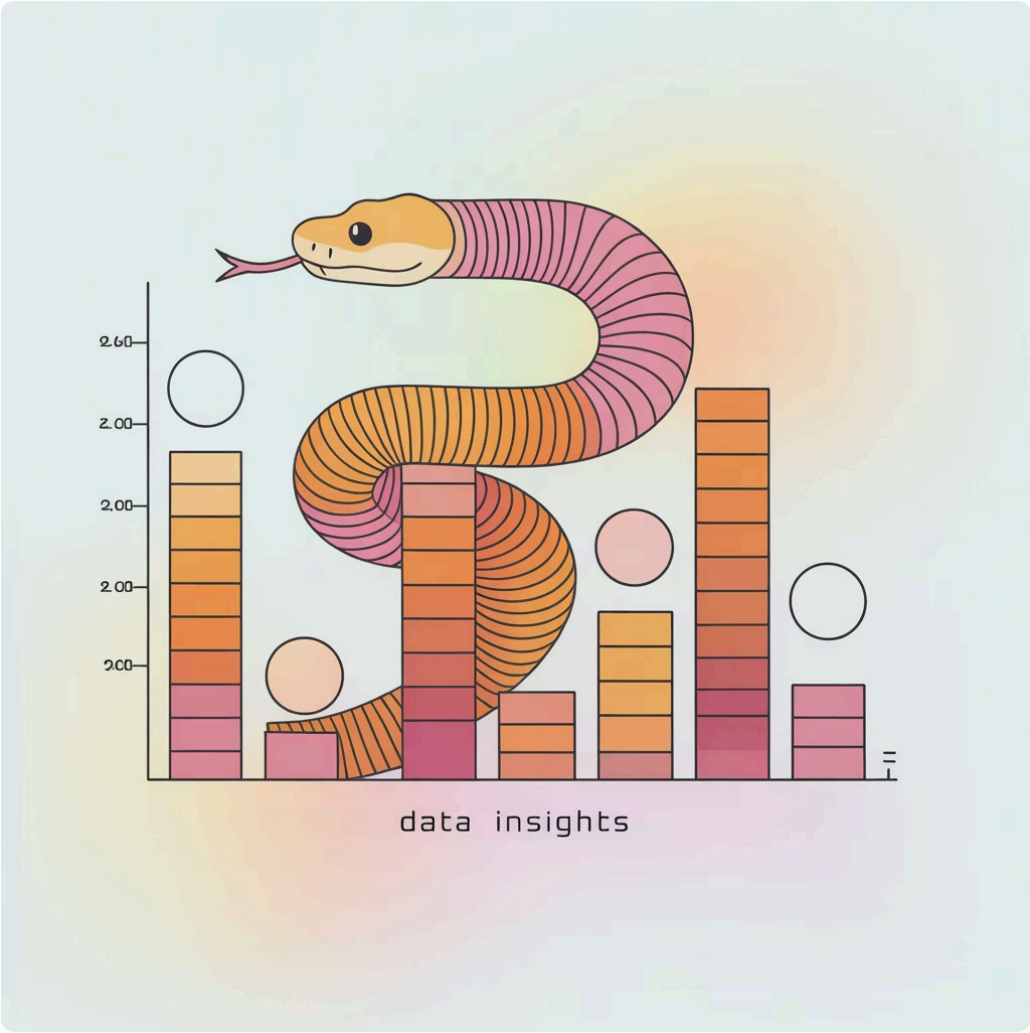
Obtiene la longitud (cantidad de elementos). *Uso DA:* Contar registros o filas.

max(lista) / min(lista)

Obtiene el elemento máximo y mínimo de la lista para análisis de rangos.

.count(valor)

Devuelve el número de veces que aparece un elemento. *Uso DA:* Frecuencia de datos categóricos.



Made with GAMMA

Tuplas: Inmutables

Sintaxis con Paréntesis

Se definen usando paréntesis ()

```
coordenadas = (40.41, -3.70)
```

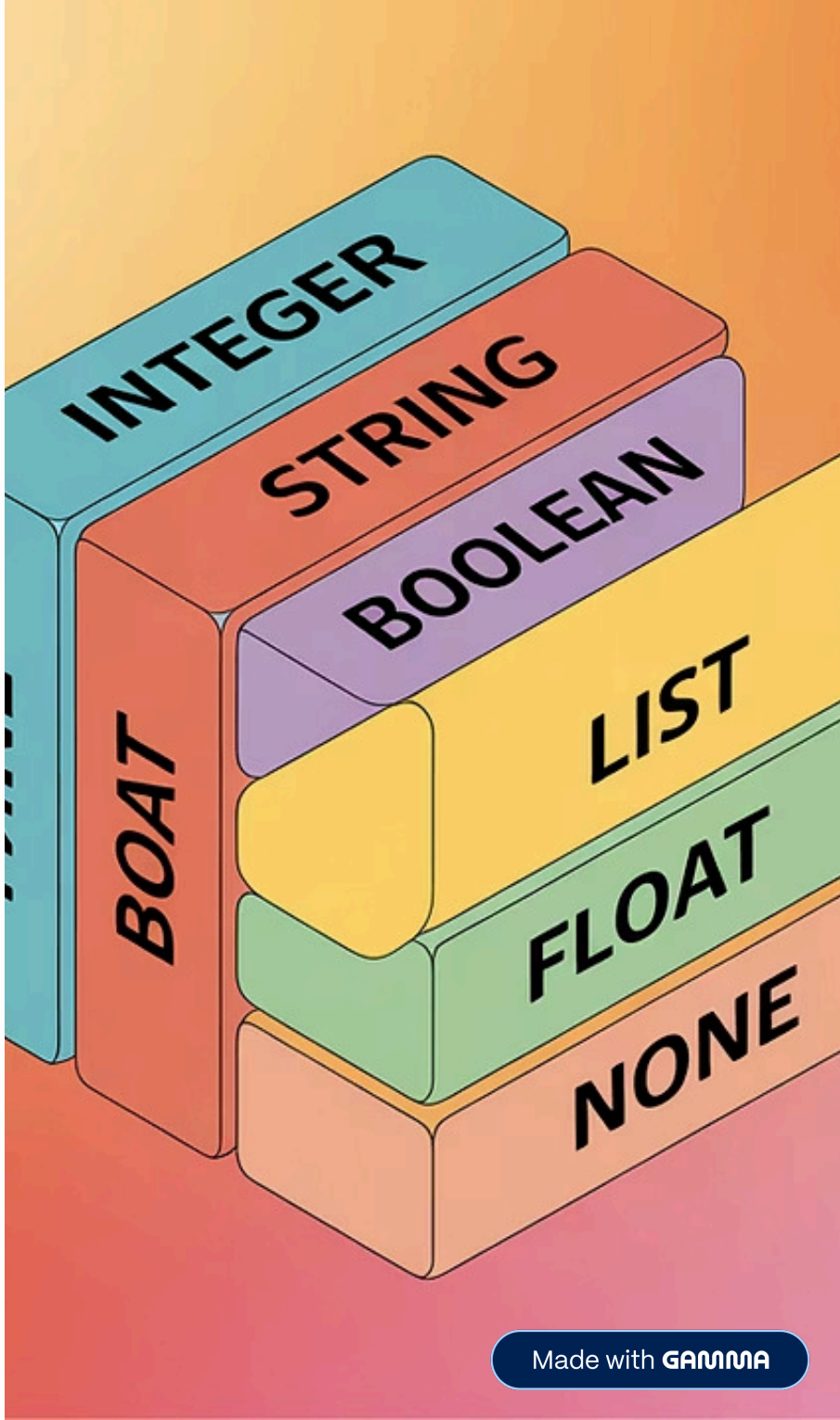
Inmutabilidad (CLAVE)

Una vez creadas, no se pueden modificar. No se puede agregar, eliminar o cambiar elementos.

Ventajas en Análisis

Integridad: Garantizan que los valores no cambien accidentalmente.
Eficiencia: Más rápidas que las listas para operaciones de lectura.

Uso ideal: Datos constantes como coordenadas, fechas de nacimiento, claves inamovibles.



Made with GAMMA

Tuplas: Creación Especial y Unpacking

Métodos Limitados

Solo tienen dos métodos: `.count()` e `.index()`

Creación de 1 Elemento

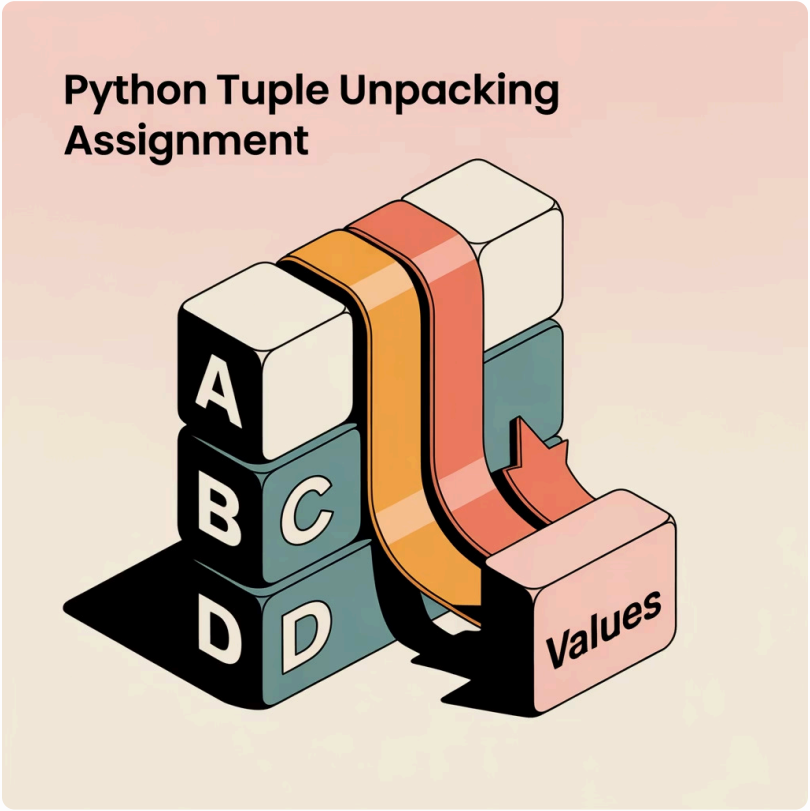
Para crear una tupla con un solo elemento, debes incluir una coma al final:

```
no_tupla = (3)    # Es un entero (int)
es_tupla = (3,)   # Es una tupla (tuple)
```

Desempaquetado (Unpacking)

Asigna los elementos de la tupla a variables individuales en una sola línea:

```
latitud, longitud = coordenadas
```



Made with GAMMA

La Función zip()

Múltiples Listas

```
nombres = ["Ana", "Beto"]
edades = [25, 30]
```

Resultado

```
list(zip(nombres, edades))
# [('Ana', 25), ('Beto', 30)]
```

Función zip()

Combina elemento por elemento en tuplas

Enfoque Análisis de Datos: Paso fundamental para juntar columnas de datos (Nombre con Edad, Producto con Precio) antes de usar estructuras más avanzadas. Nota: zip se corta automáticamente al tamaño de la lista más corta.

Made with GAMMA

Listas vs. Tuplas: ¿Cuándo Usar Cuál?

Resumen para el Analista de Datos

Característica	Listas []	Tuplas ()
Mutabilidad	Sí (Modificable)	No (Inmutable)
Uso Principal	Tareas de ETL, Limpieza, Data que necesita ser modificada	Datos Fijos, Constantes, Coordenadas, Integridad del dato
Ventaja	Gran flexibilidad de métodos	Mayor seguridad, más eficiencia en lectura

Recomendación: Usa listas para datos dinámicos y tuplas para datos que deben permanecer constantes durante el análisis.

Made with GAMMA